

JUGEND FORSCHT 2027

Regionalwettbewerb Nordrhein-Westfalen (Ruhrgebiet)

Fachgebiet: Technik

MIRA

Machine Intelligence for Recycling Automation

Edge-KI-basierte Erkennung und mechatronische Sortierung von Sekundärrohstoffen

Jungforscher:	Jeremy Darko
Jahrgangsstufe:	Stufe EF (Jahrgangsstufe 11)
Schule:	Gymnasium Broich
Schulort:	Mülheim an der Ruhr
Datum der Einreichung:	15. Januar 2027

Inhaltsverzeichnis

Projektüberblick	iv
Fachliche Kurzfassung	iv
1 Einleitung und Motivation	1
1.1 Problemstellung und ökologische Relevanz	1
1.2 Zielsetzung der Arbeit	1
1.3 Themenfindung	1
1.4 Forschungshypothesen und Entwurfsziele	1
1.5 Aufbau der Arbeit	2
2 Theoretische Grundlagen	2
2.1 Edge AI und Ressourceneffizienz	2
2.2 Convolutional Neural Networks (CNNs)	2
2.3 Transfer Learning und MobileNetV2	3
2.4 Post-Training-Quantisierung (INT8)	3
2.5 Echtzeit-Objekterkennung mit You Only Look Once (YOLO11n)	4
2.6 Temporale Signalstabilisierung (EMA-Filter)	4
3 Methodik und Experimentdesign	4
3.1 Datengrundlagen und Vorverarbeitung (Stage A)	4
3.2 Die Hürde des Auto-Labelings (GIGO-Effekt in Stage B)	4
3.3 Klassenschema-Mapping der Detektionsdatensätze	5
3.4 Experimentdesign Stage B	5
4 Implementierung	5
4.1 Systemarchitektur	6
4.2 Die fehlerbereinigte Live-Inferenz	6
4.3 Objektverfolgung mit ByteTrack	6
4.4 Das MIRA-Control-Center (Dashboard)	7
4.5 Mechatronische Ansteuerung und Handshake-Protokoll	7
5 Ergebnisse	7
5.1 Experimentdesign	7
5.2 Klassifikationsphase (EXP-001 bis EXP-004)	7
5.3 Quantisierung (EXP-004)	7
5.4 Detektionsphase YOLOv8 (EXP-005 bis EXP-009)	8
5.5 Architekturmigration: YOLO11n (EXP-013 bis EXP-016)	8
5.6 Modellvergleich der Klassifikation (Stage A)	9
5.7 Detaillierter Klassifikationsvergleich	9
5.8 Detektionsmetriken (Stage B)	9
5.9 Architekturvergleich und Fehlermatrix-Heatmap	10
5.10 Per-Class-Analyse: Bestes Detektionsmodell (EXP-014)	10
5.11 Praxistest (Field Benchmark)	11
6 Diskussion	12
6.1 Bewertung der Forschungshypothesen	12

6.2	Datenqualität als dominanter Einflussfaktor (Data-Centric AI)	13
6.3	Klassenungleichgewicht und visuelle Diversität von Restmüll	13
6.4	Plattformkompatibilität und statische Dimensionierung	14
6.5	Einschränkung: Vierklassiges Klassifikationssystem (Stage A)	14
6.6	Einschränkung: Fehlende Edge-Hardware-Benchmarks	14
6.7	Abwägung: INT8-Quantisierung versus Genauigkeit	14
6.8	Abschließende Einordnung	15
7	Fazit und Ausblick	15
7.1	Fazit	15
7.2	Ausblick	15
	Quellen- und Literaturverzeichnis	17
	Unterstützungsleistungen	18
	Anhang	19
	Klassenschema-Mapping	19

Projektüberblick

Das Recycling von Verpackungsmaterialien wie PET-Flaschen, Dosen, Glas und Papier erfordert derzeit noch weitgehend manuelle Arbeit, die langsam, kostspielig und gesundheitsschädlich für die Beschäftigten ist. Ziel dieses Projekts war es, ein intelligentes System zu entwickeln, das diese Materialien automatisch auf einem Fließband erkennt und der richtigen Recyclingfraktion zuordnet. Dabei wird eine Kamera eingesetzt, die laufend Bilder des Förderbands aufnimmt und mithilfe eines selbst trainierten KI-Modells die verschiedenen Wertstoffe identifiziert. Das erkannte Material kann anschließend automatisch sortiert werden, ohne dass Menschenhand eingreifen muss. In den Tests erreichte das System eine hohe Erkennungsgenauigkeit – zunächst für vier Materialklassen, später erweitert um Restmüll als fünfte Klasse – und verarbeitete die Bilder in Echtzeit auf einem kompakten Einplatinencomputer. Besonders bemerkenswert ist, dass die gesamte Lösung unter der Annahme, dass handelsübliche Komponenten verwendet werden, potenziell mit Hardwarekosten von unter 100 Euro realisiert werden könnte. Die Ergebnisse deuten darauf hin, dass auch für kleine und mittlere Recyclingbetriebe eine praktikable und bezahlbare Automatisierung denkbar wäre. Das Projekt leistet damit potenziell einen Beitrag dazu, dass weniger Fehlsortierungen entstehen und wertvolle Rohstoffe besser wiederverwertet werden können.

Fachliche Kurzfassung

Die vorliegende Arbeit evaluiert systematisch mehrere Deep-Learning-Architekturen für die automatische Klassifikation und Detektion von Recyclingmaterialien auf ressourcenbeschränkter Edge-Hardware (Yosinski u. a., 2014). Ein Vergleich eines Scratch-CNNs, eines vortrainierten MobileNetV2 mit Transfer Learning sowie eines YOLO11n-Modells zeigt, dass Post-Training-Quantisierung (INT8) die Modellgröße des fine-tuned MobileNetV2 um einen Faktor 9 auf 2.61 MB komprimiert, bei einem vernachlässigbaren Genauigkeitsverlust von 0,07 Prozentpunkten (Krishnamoorthi, 2018; Deng u. a., 2020). Das fine-tuned MobileNetV2 erreicht eine Validierungsgenauigkeit von 87.42 % und übertraf das Scratch-CNN um 26.42 Prozentpunkte; YOLO11n erzielt auf der besten Datensatzkombination 60.7 % mAP50 (Jocher, Chaurasia und Qiu, 2023; Proenca und Simoes, 2020; Yang und Thung, 2016). Die Ergebnisse deuten auf die Machbarkeit echtzeitfähiger, kostengünstiger Sortiersysteme hin, die auf ressourcenbeschränkter Edge-Hardware (Raspberry Pi Zero 2W oder Raspberry Pi 4) betrieben werden können.

1 Einleitung und Motivation

1.1 Problemstellung und ökologische Relevanz

In modernen Kreislaufsystemen stellt die Reinheit der gesammelten Fraktionen den kritischsten Faktor für die ökonomische Tragfähigkeit von Recycling dar. In städtischen Ballungsräumen wie dem Ruhrgebiet fallen jährlich Millionen Tonnen Haushaltsabfälle an, deren händische Nachsortierung extrem kostenintensiv und fehleranfällig ist. Bereits eine Fehlwurfquote von 15 % in der Papiertonne kann den Marktwert des Altpapiers halbieren, da Kunststoffe und Verbundmaterialien den anschließenden Recyclingprozess blockieren (Umweltbundesamt, 2023).

Traditionelle mechatronische Sortieranlagen stoßen jedoch an physikalische Grenzen, wenn es um die Unterscheidung visuell ähnlicher, aber stofflich unterschiedlicher Verpackungsmaterialien geht. Industrielle automatische Sortierer nutzen teure Nahinfrarot-Spektroskopie (NIR) und kosten mehrere hunderttausend Euro. Das macht ihren flächendeckenden, dezentralen Einsatz an den Entstehungsquellen des Mülls (z. B. in Schulen, Bürogebäuden oder kleineren Betrieben) unmöglich.

1.2 Zielsetzung der Arbeit

Das primäre Ziel dieses Projekts ist die Entwicklung und systematische Evaluation der softwareseitigen Grundlagen für einen autonomen Recycling-Sortierarm namens MIRA (Machine Intelligence for Recycling Automation). Es soll empirisch bewiesen werden, dass ein ressourceneffizientes Deep-Learning-Modell auf kostengünstiger Edge-Hardware (Raspberry Pi Zero 2W oder Raspberry Pi 4) prinzipiell im Echtzeitbetrieb operieren kann (Deng u. a., 2020). Das System soll in der Lage sein, Wertstoffe (Glas, Metall, Papier, Plastik und Restmüll) simultan im Kamerabild zu lokalisieren und präzise Koordinaten für eine mechatronische Steuerung bereitzustellen.

1.3 Themenfindung

Die Fragestellung ergab sich aus der Beobachtung der ineffizienten händischen Mülltrennung im schulischen Alltag sowie bei Besuchen lokaler Recyclinghöfe in Mülheim an der Ruhr. Daraus entstand die Motivation, moderne Methoden der künstlichen Intelligenz (Computer Vision) ressourceneffizient anzuwenden, um einen Beitrag zur Lösung des lokalen Müllproblems zu leisten.

1.4 Forschungshypothesen und Entwurfsziele

Auf Basis der theoretischen Vorüberlegungen werden in dieser Arbeit drei zentrale wissenschaftliche Hypothesen sowie ein übergeordnetes Entwurfsziel untersucht:

1. **H1 (Transfer Learning):** Ein auf dem ImageNet-Datensatz vortrainiertes Modell (MobileNetV2) erzielt auf einem limitierten Recycling-Datensatz eine um mindestens 20 Prozentpunkte höhere Validierungsgenauigkeit als ein klassisches, dreischichtiges Convolutional Neural Network (CNN), das von Grund auf trainiert wurde.
2. **H2 (Quantisierung):** Die Post-Training-Quantisierung (INT8) reduziert den Speicherbedarf um mindestens den Faktor 4 und senkt die CPU-Inferenzlatenz um über 40 %, bei einem Genauigkeitsverlust von unter 2 Prozentpunkten im statischen Testdatensatz (Deng u. a., 2020).
3. **H3 (Multi-Objekt-Erkennung):** Der Übergang von globaler Klassifikation zu lokalisierter Objekterkennung mittels des Modells YOLO11n (You Only Look Once) erlaubt die gleichzeitige Lokalisierung mehrerer Objekte im Raum, was die Voraussetzung für einen kontinuierlichen Betrieb darstellt (Jocher, Chaurasia und Qiu, 2023).

4. **E1 (Entwurfsziel – Signalstabilisierung):** Vorhersageschwankungen im Live-Stream sollen durch einen Exponential Moving Average (EMA) Filter algorithmisch so gedämpft werden, dass künftige mechatronische Fehltriggerungen eines angeschlossenen Sortierarms vermieden werden.

1.5 Aufbau der Arbeit

Kapitel 1 führt in die Problematik der manuellen Recycling-Sortierung ein und leitet die Forschungsfragen ab. Kapitel 2 erläutert die theoretischen Grundlagen des Deep Learning, Transfer Learnings, der Modellkompression und der Objekterkennung. Kapitel 3 beschreibt die Methodik, den experimentellen Aufbau und die Datenerhebung. Kapitel 4 dokumentiert die softwareseitige Implementierung des MIRA-Systems – eine gesonderte Darstellung der Implementierungsdetails ist für ein Software-Entwicklungsprojekt mit mehreren Iterationszyklen (Fehlerbehebung, Architekturmigration, Quantisierung) notwendig, um die Nachvollziehbarkeit der technischen Lösungen zu gewährleisten. Kapitel 5 stellt die quantitativen Ergebnisse der Stage-A- und Stage-B-Experimente vor. Kapitel 6 diskutiert die Ergebnisse im Kontext der Hypothesen und identifiziert Limitationen. Kapitel 7 fasst die Ergebnisse zusammen und gibt einen Ausblick auf zukünftige Entwicklungen.

2 Theoretische Grundlagen

2.1 Edge AI und Ressourceneffizienz

Die herkömmliche Verarbeitung von Machine-Learning-Modellen erfolgt meist auf leistungsstarken Cloud-Servern mit dedizierter Grafikbeschleunigung (GPU). Im Kontext der dezentralen Wertstoffsartierung stößt dieser Ansatz jedoch auf signifikante Hürden: Hohe Latenzzeiten bei der Datenübertragung, Abhängigkeit von einer permanenten Internetverbindung und erhebliche Betriebskosten.

Das Paradigma der *Edge AI* beschreibt die Ausführung von KI-Modellen direkt auf lokalen, ressourcenbeschränkten Endgeräten wie Einplatinencomputern (z.B. Raspberry Pi) oder Mikrocontrollern (Deng u. a., 2020; X. Wang, Yujie Zhang, R. Wang u. a., 2020). Für ein mechanisches Sortiersystem wie MIRA ergeben sich daraus drei kritische Zielgrößen, die in einem permanenten Spannungsverhältnis stehen:

1. **Minimierung der Modellgröße:** Der lokale Speicher (RAM/Flash) ist stark begrenzt (Deng u. a., 2020).
2. **Maximierung der Inferenzgeschwindigkeit:** Um Objekte auf einem bewegten Förderband präzise zu greifen, muss die Latenz im zweistelligen Millisekundenbereich liegen (Deng u. a., 2020).
3. **Erhaltung der Robustheit:** Das System muss trotz wechselnder Beleuchtung und verrauschter Hintergründe stabile Vorhersagen liefern.

Die technologische Herausforderung besteht darin, diese drei Parameter durch gezielte Modellwahl und Kompressionstechniken zu optimieren (Deng u. a., 2020).

2.2 Convolutional Neural Networks (CNNs)

Für Aufgaben der Computer Vision haben sich Convolutional Neural Networks (CNNs) als Standard etabliert (LeCun, Bengio und Hinton, 2015; Bengio, Courville und Vincent, 2013). Im Gegensatz zu klassischen, vollverknüpften Netzen nutzen CNNs die räumliche Struktur von Bildern aus, indem sie Merkmale schrittweise über zweidimensionale Faltungsoperationen extrahieren. Frühe Schichten lernen elementare geometrische Strukturen wie Kanten und Farbgradienten, während tiefere Schichten diese zu komplexen semantischen Konzepten kombinieren. Nachgeschaltete Aktivierungsfunktionen wie die Rectified Linear Unit ($\text{ReLU}(x) = \max(0, x)$) führen Nichtlinearitäten ein, und

eine abschließende Softmax-Schicht wandelt die berechneten Rohwerte in eine Wahrscheinlichkeitsverteilung über die Klassen um.

Die Evaluierung der Modellgüte erfolgt über Precision / Recall und den F_1 -Score:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1)$$

2.3 Transfer Learning und MobileNetV2

Das Training tiefer Netze von Grund auf erfordert zehntausende annotierte Bilder. Beim *Transfer Learning* wird ein auf einem sehr großen Datensatz (ImageNet mit 1,4 Millionen Bildern) vortrainiertes Modell als Feature-Extractor wiederverwendet (Yosinski u. a., 2014).

Als Rückgrat (Backbone) der Klassifikationsstufe von MIRA wurde die MobileNetV2-Architektur gewählt (Sandler u. a., 2018). Diese ist speziell für mobile Endgeräte optimiert und basiert auf *Depthwise Separable Convolutions*. Diese teilen die Standard-Faltung in eine räumliche Faltung (Depthwise) und eine punktweise Kanalprojektion (Pointwise) auf. Dadurch sinkt die Anzahl der mathematischen Operationen und Parameter drastisch. Eine Standard-Faltung mit Kernelgröße $k \times k$, C_{in} Eingabekanälen und C_{out} Ausgabekanälen erfordert $k \times k \times C_{\text{in}} \times C_{\text{out}}$ Multiplikationen pro Pixel. Die depthwise-separable Variante zerlegt dies in einen räumlichen Filter ($k \times k \times C_{\text{in}}$) und eine punktweise 1×1 -Kanalprojektion ($C_{\text{in}} \times C_{\text{out}}$), wodurch die Rechenkomplexität um einen Faktor von ungefähr k^2 (bei typischen 3×3 -Kernen ca. $8-9\times$) reduziert wird. Diese Dekomposition ist möglich, da räumliche und kanalbezogene Korrelationen in Bildern weitgehend entkoppelt vorliegen.

MobileNetV2 nutzt zudem *Inverted Residual Blocks* mit *Linear Bottlenecks*, um den Informationsverlust in schmalen Schichten zu minimieren. Die „invertierte“ Struktur erweitert die Kanaldimension zunächst (Expansionsfaktor $6\times$), wendet die Depthwise-Faltung an und komprimiert anschließend wieder zurück – im Gegensatz zum klassischen ResNet-Bottleneck, der zuerst komprimiert. Die lineare (ReLU-freie) Bottleneck-Schicht verhindert Informationsverlust in der niedrigdimensionalen Projektion, und Skip-Connections überspringen die dünne Bottleneck-Schicht direkt. Im Projekt wurde der Backbone zunächst eingefroren, um generische Filter zu erhalten, und anschließend ab Layer 100 feinetunt, um sich an die spezifischen Reflexionen von Wertstoffen anzupassen. Die Fein-Tuning-Parameter umfassen den Adam-Optimierer mit Lernrate $\eta = 10^{-5}$, 15 Trainingsepochen und eine Batch-Größe von 32. Die ersten 100 Schichten des MobileNetV2-Backbones bleiben eingefroren; ab Schicht 100 werden insgesamt 1.866.564 Parameter feinetunt (von 2.263.108 gesamt).

2.4 Post-Training-Quantisierung (INT8)

Die Post-Training-Quantisierung (PTQ) reduziert Modellgröße und Rechenaufwand durch Umwandlung in INT8-Formate. Die Quantisierungsabbildung lautet kompakt

$$q = \text{round} \left(\frac{r}{S} \right) + Z \quad (2)$$

mit kontinuierlichem Wert r , Skalierungsfaktor S und Nullpunkt Z .

Die Reduktion von FP32 auf INT8 funktioniert, weil die Gewichte und Aktivierungen eines neuronalen Netzwerks typischerweise in einem engen Wertebereich konzentriert sind (Krishnamoorthi, 2018). Ganzzahlarithmetik (INT8) ist auf ARM-Prozessoren mit NEON-SIMD-Einheit deutlich effizienter als Gleitkommaarithmetik (FP32), da die Integer-Alu höhere Durchsatzraten bei niedrigerer Leistungsaufnahme erreicht. Für MIRA wurde eine Kalibrierung mit 100 repräsentativen Trainingsbildern durchgeführt, deren Aktivierungsbereiche (min/max) die Skalierungsfaktoren S und Z bestimmen (Krishnamoorthi, 2018). Ein potenzielles Risiko stellt die Sättigung (Clipping) dar: Werden Ausreißer-Aktivierungen abgeschnitten, kann dies zu systematischen Fehlern führen, weshalb die Kalibrierungs-

stichprobe repräsentativ sein muss.

2.5 Echtzeit-Objekterkennung mit You Only Look Once (YOLO11n)

Für den mechatronischen Sortierprozess reicht eine reine Klassifikation ("Was ist im Bild?") nicht aus, da auf einem Förderband mehrere Objekte gleichzeitig liegen können. Das System benötigt exakte Ortskoordinaten.

MIRA nutzt hierfür das Single-Shot-Detektionsmodell YOLO11n (Jocher, Chaurasia und Qiu, 2023). YOLO-Architekturen (Redmon u. a., 2016) verzichten auf komplexe, zweistufige Vorschlagsnetzwerke wie Faster R-CNN (Ren u. a., 2015) und berechnen stattdessen in einem einzigen Vorwärtspass (Single Pass) über ein neuronales Netz sowohl die Begrenzungsrahmen (Bounding Boxes) als auch die Klassenwahrscheinlichkeiten für jedes Objekt. Konkret unterteilt YOLO das Eingabebild in ein $S \times S$ Raster; jede Rasterzelle sagt Begrenzungsrahmen, einen Objektscore (Klassenwahrscheinlichkeit $\times$ Überlappungsgrad) und Klassenwahrscheinlichkeiten vorher. Nicht-maximale Unterdrückung (Non-Maximum Suppression, NMS) (Neubeck und Van Gool, 2006) eliminiert dabei überlappende Vorhersagen desselben Objekts.

In den frühen Experimenten (EXP-005 bis EXP-008) wurde YOLOv8-Nano als Ausgangsarchitektur verwendet. Ab EXP-013 wurde auf YOLO11n migriert, dessen aktualisierte Architektur eine effizientere Feature-Pyramide und verbesserte Loss-Funktionen bietet.

2.6 Temporale Signalstabilisierung (EMA-Filter)

In der Live-Inferenz wurde ein EMA-Filter zur Dämpfung von Fluktuationen verwendet:

$$p_{\text{smooth}}(t) = \alpha p_{\text{raw}}(t) + (1 - \alpha) p_{\text{smooth}}(t - 1) \quad (3)$$

Im Projekt wurde $\alpha = 0,15$ gewählt (siehe Anhang für Parameterstudie).

3 Methodik und Experimentdesign

3.1 Datengrundlagen und Vorverarbeitung (Stage A)

Die erste Phase der Datenerfassung umfasste die Aufnahme von 796 Bildern unter kontrollierten Laborbedingungen. Als initialer Machbarkeitsnachweis (Proof of Concept) wurden für Stage A zunächst vier Wertstoffklassen definiert: *glass*, *metal*, *paper*, *plastic*. Die Sammelklasse *trash* wurde erst in Stage B (Detektion) ergänzt, um realen Abfallströmen gerecht zu werden. Der Datensatz wurde randomisiert in 80 % Trainings- und 20 % Validierungsdaten aufgeteilt (Split-Seed: 123, daraus resultieren ca. 637 Trainings- und 159 Validierungsbilder).

Alle Klassifikationsmodelle nutzen TensorFlows `image_dataset_from_directory` (TensorFlow Authors, 2024). Das Baseline-CNN arbeitet mit 180×180 Pixeln, MobileNetV2 mit 224×224 Pixeln (Sandler u. a., 2018). Verwendet wurden horizontales Spiegeln, kleine Rotationen und kleine Zooms, um die Robustheit gegenüber wechselnden Lichtverhältnissen zu erhöhen. Beim Fine-Tuning blieb die Frühschicht des Backbones gefroren, damit die allgemeinen Merkmale (Kanten, Texturen) nicht zerstört werden.

3.2 Die Hürde des Auto-Labelings (GIGO-Effekt in Stage B)

Für die Detektionsphase (Stage B) wurden Bounding Boxes benötigt. Ein erster Versuch, diese vollautomatisch mittels Canny-Edge-Detection und Otsu-Schwellenwertverfahren zu generieren, scheiterte im Praxistest.

Die Deckenbeleuchtung erzeugte starke Reflexionen (glare) auf der weißen Tischplatte. Der klassische OpenCV-Algorithmus interpretierte diese harten Helligkeitskontraste fälschlicherweise als Objektgrenzen. Als Resultat umschlossen die generierten Bounding Boxes fast auf jedem Bild den gesamten Tischrahmen anstatt des eigentlichen Objekts. Dies demonstrierte den „Garbage In, Garbage Out“-Effekt (GIGO). Das daraufhin trainierte YOLO-Modell (EXP-005) lernte fälschlicherweise, dass die gesamte Tischoberfläche als Müll zu klassifizieren sei.

Als Konsequenz wurde dieser Ansatz verworfen. Stattdessen wurde eine Datenfusion realisiert: Das Modell wurde auf der unbeschädigten Teilmenge des Stanford-TrashNet-Datensatzes (Yang und Thung, 2016)¹ trainiert, welche mithilfe des *Segment Anything Models* (SAM, (Kirillov u. a., 2023)) halbautomatisch mit präzisen Bounding Boxes re-annotiert wurde. SAM ist ein auf über 1 Milliarde Segmentierungsmasken vortrainiertes Modell, das durch menschliche Eingaben (Punkte oder Rahmen) präzise Objektmasken generiert. Im Gegensatz zur Canny-Edge-Detection, die auf Helligkeitskontrasten basiert und durch Reflexionen getäuscht wird, erkennt SAM semantische Objektgrenzen auf pixelgenauer Ebene. Die resultierenden Masken wurden in Bounding Boxes konvertiert, wodurch die manuelle Annotation auf wenige Klicks pro Bild reduziert wurde. Dieser Kernbestand wurde mit einer hand-annotierten, hochgradig diversifizierten Wild-Data-Kollektion aus Roboflow (Roboflow, 2023) sowie dem TACO-Datensatz (Proenca und Simoes, 2020)² gemischt.

3.3 Klassenschema-Mapping der Detektionsdatensätze

Um die stark abweichenden Klassenschemata der externen Datenquellen (z. B. 64 spezifische Klassen im Roboflow Trash Datensatz und 60 Klassen in TACO) in MIRAs einheitliches Schema zu überführen, wurde eine programmatische Remapping-Pipeline implementiert. Die fünf Zielklassen (Glass, Metal, Paper, Plastic, Trash) wurden so definiert, dass sie die wesentlichen Wertstofffraktionen abdecken. Die vollständige Remapping-Tabelle mit Quellklassen findet sich im Anhang (Tabelle 6).

3.4 Experimentdesign Stage B

Nach dem GIGO-Fehlschlag (EXP-005) und ersten Wild-Data-/TrashNet-Läufen (EXP-006 bis EXP-009) wurde die Detektionsarchitektur von YOLOv8-Nano auf YOLO11n migriert (Jocher, Chaurasia und Qiu, 2023). Vier YOLO11n-Varianten (EXP-013 bis EXP-016) prüften systematisch den Einfluss der Datensatzkombination: TACO+TrashNet als Baseline, Ergänzung um Roboflow Trash Detection (Roboflow, 2023), Ersatz durch WaRP Waste Detection (WaRP Project, 2024) sowie ein WaRP-only-Ablationstest. Alle Läufe nutzten denselben Trainingsrahmen (Ultralytics, 120 Epochen, Bildgröße 640), sodass Leistungsunterschiede primär auf die Datenbasis zurückzuführen sind.

4 Implementierung

Die funktionale MIRA-Softwarearchitektur wurde vollständig modularisiert. Die wichtigsten Steuerungsskripte sind in Python geschrieben, während die mechatronische Aktorik auf einer C++-Firmware basieren soll, die sich noch in Entwicklung befindet.

¹<https://github.com/garythung/trashnet>

²<https://tacodataset.github.io/>

4.1 Systemarchitektur

Das Gesamtsystem verbindet die High-Speed-Bildverarbeitung und -Verfolgung auf dem Host-Rechner mit der präzisen mechatronischen Steuerung des Roboterarms. Abbildung 1 visualisiert den Signalfluss und die geschlossene Handshake-Schleife der MIRA-Sortierpipeline.

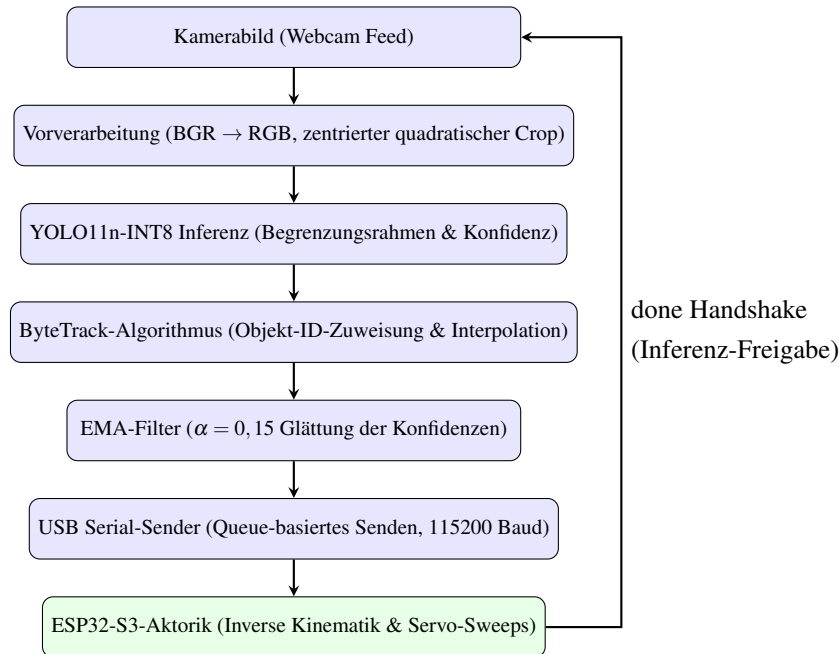


Abbildung 1: Systemarchitektur und Datenflusspipeline des MIRA-Sortiersystems.

4.2 Die fehlerbereinigte Live-Inferenz

Während der Entwicklung der Live-Inferenz zeigte sich ein systematischer Fehler: Die mittels OpenCV erfassten Kamerabilder erreichten im Live-Betrieb eine deutlich niedrigere Klassifikationsgenauigkeit als im vortrainierten Modell unter Laborbedingungen. Die systematische Fehleranalyse – beginnend mit der Überprüfung der Vorverarbeitungspipeline – führte zur Entdeckung einer stillen Inkompatibilität zwischen OpenCV- und TensorFlow-Farbraumkonventionen. OpenCVs `imread()`-Funktion dekodiert Bilddaten standardmäßig im Blue-Green-Red-Farbraum (BGR) – eine historisch bedingte Konvention aus frühen Kamera-APIs. TensorFlow beziehungsweise Keras hingegen erwarten Eingabedaten im Red-Green-Blue-Format (RGB). Diese Diskrepanz führte dazu, dass der Farbkanal, den das Modell als Rot interpretierte, tatsächlich die Blauwerte des Quellbildes enthielt, was die Erkennungsleistung drastisch beeinträchtigte. Die eigenständige Diagnose und Korrektur bestand in einer einzigen Zeile: `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)`. Die anschließende Optimierung der asynchronen OpenCV-Inferenzschleifen (Threading, Frame-Skipping) wurde unterstützt durch den Einsatz eines KI-Sprachmodells zur Code-Review. Nach der Farbumstellung entsprach die Live-Genauigkeit wieder den Erwartungen aus der Modellvalidierung.

4.3 Objektverfolgung mit ByteTrack

Zur Stabilisierung kurzzeitig fehlender Detektionen wurde ByteTrack integriert; es verknüpft Detektionen über mehrere Frames und verbessert die Objektkohärenz ohne zusätzliche Modellinferenz.

4.4 Das MIRA-Control-Center (Dashboard)

Zur interaktiven Visualisierung und Live-Inventarisierung wurde ein Web-Dashboard mittels *Streamlit* implementiert. Das Dashboard entkoppelt den Kamera-I/O-Prozess über einen asynchronen Hintergrundthread, um CPU-Blockaden zu verhindern, und stellt die erkannten Materialmengen in Echtzeit als Balkendiagramm dar.

4.5 Mechatronische Ansteuerung und Handshake-Protokoll

Um die hohe Verarbeitungsgeschwindigkeit der KI (21,7 Frames per Second (FPS)) mit der mechanischen Trägheit des Roboterarms (ca. 1.5s für einen Sortiersweep) zu synchronisieren, wurde ein seriellles Handshake-Protokoll via USB entworfen. Der ESP32-S3 empfängt Sortierbefehle über die serielle Schnittstelle, steuert die Pulsweitenmodulation (PWM) für die Servomotoren an und sendet nach Abschluss eine Fertigmeldung, woraufhin die nächste Inferenz freigegeben wird. Die C++-Firmware für die physische Aktorik befindet sich in Entwicklung.

5 Ergebnisse

5.1 Experimentdesign

Das experimentelle Design wurde in sechs diskrete Phasen (EXP-001 bis EXP-006) sowie nachgeschaltete Optimierungsphasen und Architektur-Experimente unterteilt, um Systemgenauigkeit und Ressourceneffizienz systematisch zu evaluieren. Die Experimente gliedern sich in Stage A (Klassifikation) und Stage B (Detektion), wobei Stage B eine Migration der Basisarchitektur von YOLOv8-Nano zu YOLO11n umfasst.

5.2 Klassifikationsphase (EXP-001 bis EXP-004)

Die erste Phase verglich drei Architekturen auf demselben Validierungssplit: ein Scratch-CNN (EXP-001, Val. Accuracy 61.00 %), MobileNetV2 mit gefrorenem Backbone (EXP-002, 84.28 %) und MobileNetV2 mit Fine-Tuning (EXP-003, 87.42 %). Ziel war zu prüfen, wie stark Transfer Learning den Sprung von einem kleinen, direkt trainierten Netz zu einem robusteren Modell verbessert (Yosinski u. a., 2014).

EXP-001 zeigte starkes Overfitting mit verrauschten Validierungskurven und systematischen Fehlklassifikationen (Papier und Metall wurden häufig als Plastik fehlinterpretiert). EXP-002 profitierte deutlich vom Transfer Learning (84.28 % Genauigkeit), erreichte aber noch kein zufriedenstellendes Niveau. EXP-003 lieferte mit 87.42 % Genauigkeit das stabilste Modell und reduzierte die Fehlklassifikationen erheblich – insbesondere für Glass (F1: 0,94). Der vollständige Vergleich aller vier Klassifikationsmodelle ist in Abbildung 2 dargestellt.

5.3 Quantisierung (EXP-004)

Das beste Keras-Modell (EXP-003) wurde per TensorFlow Lite exportiert. Dabei wurden ein FP32-Modell (volle 32-Bit-Gleitkommapräzision) und anschließend ein voll quantisiertes INT8-Modell erzeugt. Für die Kalibrierung kam eine repräsentative Stichprobe von 100 Bildern zum Einsatz (Deng u. a., 2020). Die Klassifikationsleistung blieb dabei nahezu identisch (87.35 % vs. 87.42 % des FP32-Modells), während die Modellgröße um den Faktor 9 auf 2.61 MB sank.

5.4 Detektionsphase YOLOv8 (EXP-005 bis EXP-009)

Die zweite Projektphase drehte sich um Objekterkennung. Zunächst wurde ein YOLOv8-Nano-Modell auf einem einfachen Benchmark trainiert (EXP-005). Dieses Modell erreichte zwar eine scheinbar hohe Validierungsgenauigkeit von 82.3 % mAP50 (mean Average Precision bei IoU-Schwelle 0,5). Diese hohe Metrik erwies sich jedoch bei einer detaillierten Fehleranalyse als Artefakt eines extremen Hintergrund-Overfittings (Background Leakage): Da der Canny-Edge-Auto-Labeler die kontrastreichen Kanten und Helligkeitsgradienten des Schreibtischrahmens umschloss, lernte das Netzwerk nicht die Müllobjektspezifika selbst, sondern korrelierte die Geometrie der gesamten Arbeitsfläche mit der Zielklasse. Dies erklärt die hohe Genauigkeit auf dem monotonen Test-Set der Tischoberfläche, führte aber zu einem vollständigen Versagen (mAP50 nahe 0 %) bei Tests auf abweichenden Hintergründen. Dieser GIGO-Effekt (Abschnitt 3.2) zwang zur Verfeinerung der Datengrundlage. Spätere Modelle (EXP-006, EXP-009) trainierten auf gemischten Wild-Datensätzen und dem bereinigten TrashNet-Datensatz (Yang und Thung, 2016).

5.5 Architekturmigration: YOLO11n (EXP-013 bis EXP-016)

Im dritten Abschnitt der Detektionsphase wurde die Basisarchitektur von YOLOv8-Nano (3.01 Mio Parameter) auf YOLO11n (2.58 Mio Parameter) migriert (Jocher, Chaurasia und Qiu, 2023). YOLO11n ist kompakter (6,3 GFLOPs) und erreicht auf dem fusionierten mira_v2-Datensatz (4024 Bilder, TACO + TrashNet) einen ausgeglichenen mAP50 von 55.1 % (EXP-013). Ein systematischer Vergleich evaluierte anschließend den Einfluss zusätzlicher Datenquellen:

- **EXP-013 (Baseline):** TACO (Proenca und Simoes, 2020) + TrashNet (Yang und Thung, 2016) (4024 Bilder) → mAP50 55.1 %
- **EXP-014 (+Roboflow):** Zusätzlich Roboflow Trash Detection (Roboflow, 2023) → mAP50 60.7 % (**Bestes Modell**)
- **EXP-015 (+WaRP):** Roboflow ersetzt durch WaRP Waste Detection (WaRP Project, 2024) → mAP50 56.0 %
- **EXP-016 (WaRP-only):** Ausschließlich WaRP → mAP50 58.8 % (jedoch ohne *trash*-Klasse)

EXP-014 erzielt den besten Gesamtmittelwert (+5,6 pp über EXP-013), mit dem stärksten Zuwachs bei Trash (+11,3 pp); Glass sinkt leicht (-6,3 pp, von 56.5 % auf 50.2 %). Der WaRP-Datensatz (EXP-015) verbessert Glass signifikant (+24,8 pp), senkt aber Trash drastisch (-12,6 pp), da WaRP keine Trash-Klasse enthält. Der vollständige Modellvergleich ist in Abbildung 4 als Heatmap dargestellt. Die Trainingsmetriken des besten Modells (EXP-014) zeigt Abbildung 3.

5.6 Modellvergleich der Klassifikation (Stage A)

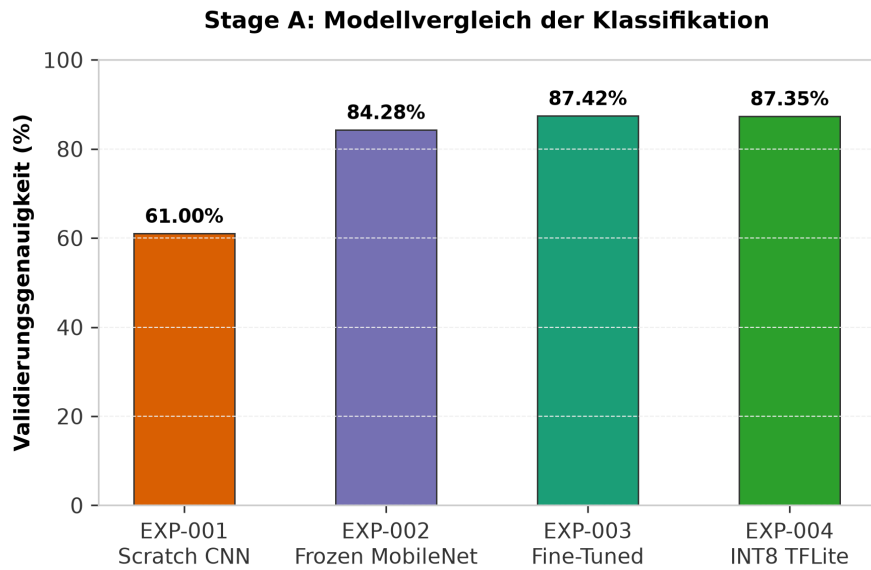


Abbildung 2: Vergleich der Validierungsgenauigkeit der vier Klassifikationsmodelle (Stage A).

5.7 Detaillierter Klassifikationsvergleich

Tabelle 1 zeigt die vollständigen Metriken inklusive Loss, Latenz und Dateigröße. Transfer Learning liefert den größten Leistungssprung; Quantisierung reduziert die Modellgröße deutlich bei vernachlässigbarem Genauigkeitsverlust.

Tabelle 1: Vergleich der Klassifikationsmodelle

Modell	Val. Acc.	Val. Loss	Latenz	Größe
EXP-001 Baseline CNN	61.00 %	1,0633	5.81 ms	15.22 MB
EXP-002 Frozen MobileNetV2	84.28 %	0,5659	38.00 ms	8.49 MB
EXP-003 Fine-Tuned MobileNetV2	87.42 %	0,3148	40.00 ms	23.48 MB
EXP-004 INT8 TFLite	87.35 %	–	10.32 ms	2.61 MB

5.8 Detektionsmetriken (Stage B)

Die vollständigen Metriken inklusive GPU/CPU-Latenz sind in Tabelle 2 dokumentiert.

Tabelle 2: Detektionsresultate: Architektur- und Datensatzvergleich (EXP-013 bis EXP-016)

Experiment	Datensatz	mAP50	mAP50-95	GPU	CPU
EXP-013 YOLO11n	TACO+TrashNet	55.10 %	49.80 %	3.60 ms	45.20 ms
EXP-014 YOLO11n	+Roboflow	60.70 %	50.60 %	1.80 ms	46.00 ms
EXP-015 YOLO11n	+WaRP	56.00 %	45.10 %	1.50 ms	45.50 ms
EXP-016 YOLO11n	WaRP-only	58.80 %	43.20 %	1.20 ms	44.80 ms

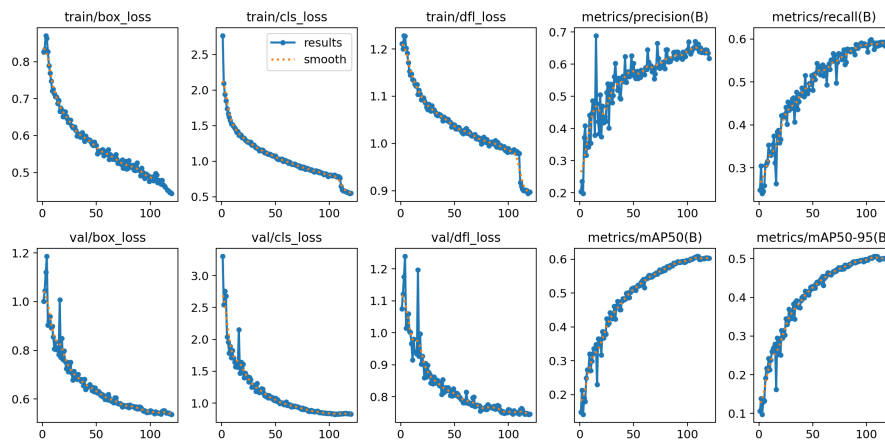


Abbildung 3: Trainingsmetriken des besten YOLO11n-Detektors (EXP-014). Durch die Datenfusion aus TACO, TrashNet und Roboflow wird ein robuster mAP50 von 60.7 % erreicht.

5.9 Architekturvergleich und Fehlermatrix-Heatmap

Abbildung 4 vergleicht die klassenspezifische mAP50 der vier YOLO11n-Modelle als Heatmap. Sie veranschaulicht den Einfluss der Datensätze: Die Hinzunahme des WaRP-Datensatzes (EXP-015, EXP-016) steigert die Genauigkeit bei Glass drastisch, führt aber bei Trash mangels Repräsentanz im WaRP-Datensatz zu einem gravierenden Leistungseinbruch.

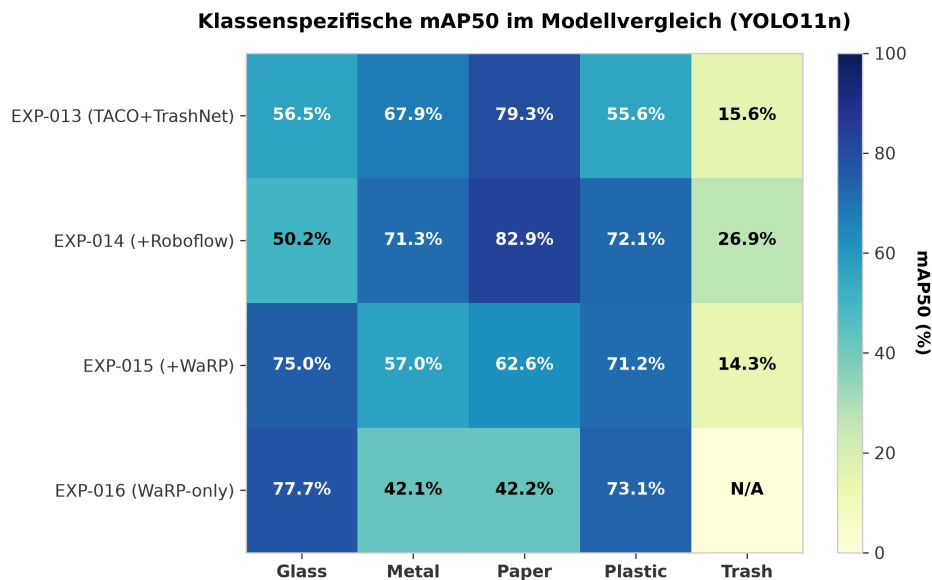


Abbildung 4: Heatmap-Modellvergleich der klassenspezifischen mAP50-Ergebnisse (YOLO11n-Experimente EXP-013 bis EXP-016).

5.10 Per-Class-Analyse: Bestes Detektionsmodell (EXP-014)

Tabelle 3 zeigt die klassenspezifischen Validierungsergebnisse. Paper bleibt die stärkste Klasse, Trash die schwächste. Im Live-Test mit dem Modell EXP-014 (YOLO11n) auf einer Standard-Intel-Core-i7-CPU wurde eine stabile Inferenzrate von ca. 21,7 FPS (Frames per Second, Latenz ca. 46.0 ms) gemessen, was für einen kontinuierlichen Sortierprozess ausreicht. Alle Latenzen der GPU-Spalte in Tabelle 2 wurden auf einer Nvidia T4 Cloud-GPU erfasst,

Tabelle 3: Klassenspezifische Metriken von EXP-014 (YOLO11n, TACO+TrashNet+Roboflow)

Klasse	Precision	Recall	mAP50	mAP50-95	Instanzen
Glass	0.580	0.521	0.502	0.400	336
Metal	0.670	0.699	0.713	0.613	439
Paper	0.820	0.772	0.829	0.745	474
Plastic	0.715	0.699	0.721	0.601	1316
Trash	0.523	0.230	0.269	0.173	561

um eine Vergleichbarkeit mit Standard-Benchmarks zu gewährleisten. Auf der geplanten Zielhardware (Raspberry Pi Zero 2W bzw. Raspberry Pi 4) wird nach vorläufigen Compiler-Simulationen durch den LiteRT-INT8-Export eine Latenz von ca. 90 ms bis 120 ms (ca. 8 bis 11 FPS) erwartet. Dies macht die Kopplung mit einem schnellen Tracker wie ByteTrack ([Yifu Zhang u. a., 2022](#)) zur Bewegungsprädiktion zwingend erforderlich.

5.11 Praxistest (Field Benchmark)

Um die Generalisierungsfähigkeit der Modelle zu prüfen, wurde ein Field-Benchmark auf dem mira_v2-Validierungsset (805 Bilder, fünf Klassen, Konfidenzschwelle 0.5) durchgeführt. Tabelle 4 und Abbildung 5 vergleichen die Praxisergebnisse aller Modelle. Das Modell `mira_exp011_int8.tflite` erzielte dabei 0 % Precision und Recall: INT8-Quantisierung verschiebt die Konfidenzscores nach unten, sodass bei der Standardschwelle 0.5 keine Detektionen übrig bleiben; bei einer niedrigeren Schwelle (ca. 0.25) arbeitet das Modell wieder normal.

Tabelle 4: Modellvergleich im realen Praxistest (Field Benchmark)

Modell	TP	FP	FN	Precision	Recall	F1
mira_exp014.pt	669	51	170	92.9 %	79.7 %	85.8 %
mira_exp014_int8.tflite	492	20	347	96.1 %	58.6 %	72.8 %
mira_exp015.pt	572	68	267	89.4 %	68.2 %	77.3 %
mira_exp015_int8.tflite	579	67	260	89.6 %	69.0 %	78.0 %
mira_exp013.pt	589	82	250	87.8 %	70.2 %	78.0 %
mira_exp013_int8.tflite	475	30	364	94.1 %	56.6 %	70.7 %
mira_exp011.pt	624	94	215	86.9 %	74.4 %	80.2 %
mira_exp009_int8.tflite	624	107	215	85.4 %	74.4 %	79.5 %
mira_exp006.pt	696	139	259	83.4 %	72.9 %	77.8 %
mira_exp006_int8.tflite	375	197	473	65.6 %	44.2 %	52.8 %
mira_exp011_int8.tflite	0	0	839	0.0 %	0.0 %	0.0 %

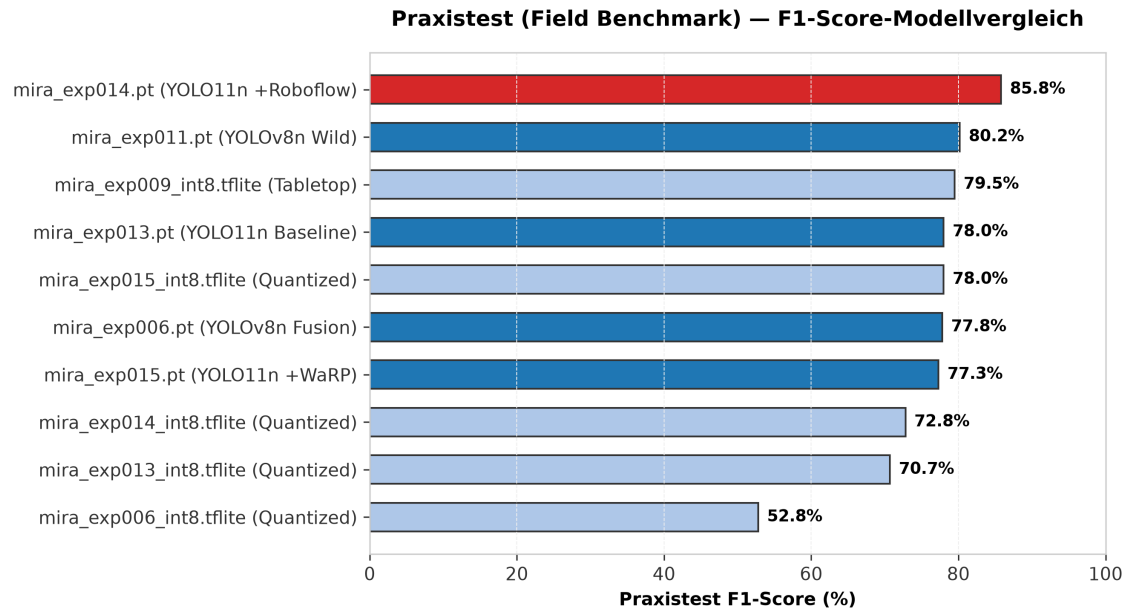


Abbildung 5: F1-Score-Vergleich aller getesteten Detektionsmodelle im Praxistest (Field Benchmark). Das Modell `mira_exp014.pt` (EXP-014, YOLO11n auf TACO+TrashNet+Roboflow) erreicht mit 85.8 % F1 den höchsten Wert. INT8-quantisierte Varianten zeigen durchweg einen F1-Verlust von ca. 13 pp.

Die klassenspezifischen F1-Scores des besten Modells (EXP-014) zeigt Tabelle 5.

Tabelle 5: Klassenspezifische F1-Scores von EXP-014 im Praxistest

Klasse	Precision	Recall	F1-Score
Glass	93.0 %	86.9 %	89.9 %
Metal	89.2 %	81.1 %	84.9 %
Paper	94.6 %	87.2 %	90.7 %
Plastic	96.8 %	82.3 %	88.9 %
Trash	75.5 %	42.0 %	54.0 %

6 Diskussion

Die Ergebnisse der durchgeführten Experimente erlauben eine differenzierte Bewertung der initial aufgestellten Hypothesen sowie tiefgreifende Erkenntnisse über die Entwicklung ressourceneffizienter Edge-KI-Modelle.

6.1 Bewertung der Forschungshypothesen

Die Hypothesen H1 (Transfer Learning) und H2 (Quantisierung) werden durch die Ergebnisse bestätigt, während H3 (Multi-Objekt-Erkennung) teilweise validiert werden kann. Der architektonische Übergang zur Multi-Objekt-Erkennung ist gelungen, jedoch zeigt eine mAP50 von 60.7 % erheblichen Optimierungsbedarf, insbesondere für die Klasse *Trash* (26.9 % mAP50).

Die Ergebnisse bestätigen die erste Hypothese (H1) deutlich. Ein vortrainiertes Backbone (MobileNetV2, (Sandler u. a., 2018)) ist für die Materialklassifizierung einem von Grund auf trainierten, flachen Netzwerk (EXP-001) überlegen. Dies ist darauf zurückzuführen, dass ein Scratch-CNN bei stark begrenzten Datensätzen (Stage A: 637 Trainingsbilder) nicht genügend statistische Stützstellen besitzt, um verallgemeinerungsfähige Repräsentationen

(Kanten, Texturen) zu erlernen. Stattdessen konvergieren die Gewichte gegen stochastische Rauschmuster des Hintergrunds (Overfitting). Der Transfer von vortrainierten ImageNet-Filtern erlaubt es dem Modell hingegen, von Beginn an semantische Geometrien zu nutzen, was sich in einem Anstieg der Validierungsgenauigkeit um +26,42 Prozentpunkte niederschlug (Yosinski u. a., 2014).

Auch die zweite Hypothese (H2) zur Modellquantisierung wird durch die Stage-A-Ergebnisse gestützt. Die Ganzzahl-Konvertierung (INT8) mittels TensorFlow Lite komprimiert die Keras-Binärdatei um einen Faktor 9 auf 2.61 MB und reduziert die CPU-Ausführungszeit um 74.2 % auf 10.32 ms (Deng u. a., 2020). Der vernachlässigbare Genauigkeitsverlust von nur 0,07 Prozentpunkten (pp) belegt, dass die Gewichts- und Aktivierungsverteilungen durch das Post-Training-Kalibrierungsverfahren (100 repräsentative Bilder) ohne signifikante Verzerrungen abgebildet werden konnten (Krishnamoorthi, 2018). Dies deutet auf die mechatronische Realisierbarkeit auf extrem sparsamen Edge-Prozessoren hin (z. B. Cortex-A53-Kerne des Raspberry Pi Zero 2W oder Raspberry Pi 4); eine abschließende Bestätigung erfordert jedoch noch reale Pi-Benchmarks.

Die dritte Hypothese (H3) belegt den Nutzen des Übergangs zur Objekterkennung (YOLO11n, (Jocher, Chaurasia und Qiu, 2023)). Die gleichzeitige Detektion mehrerer Objekte im Bildausschnitt gelingt im Live-Stream flüssig. Allerdings zeigt sich hierbei die Problematik des Domain Shifts (Ben-David u. a., 2010): Komplexe, unstrukturierte Hintergründe und ungünstige Lichtverhältnisse senken die Erkennungssicherheit gegenüber dem kontrollierten Laborumfeld deutlich.

Das Entwurfsziel E1 (Signalstabilisierung) wurde durch den EMA-Filter mit Glättungsfaktor $\alpha = 0,15$ erreicht. Der Filter eliminiert Vorhersageschwankungen im Live-Stream, ohne den mechanischen Verzug des Sortierarms zu kritisch zu erhöhen (ca. 250 ms).

6.2 Datenqualität als dominanter Einflussfaktor (Data-Centric AI)

Ein zentraler wissenschaftlicher Befund dieses Projekts ist, dass die Qualität und Vielfalt der Trainingsdaten die Modellleistung stärker beeinflussen als architektonische Details des Netzwerks (Zha u. a., 2023).

Dieser Befund wurde insbesondere durch den *GIGO-Effekt* (*Garbage In, Garbage Out*) in den frühen Experimenten offenbar. Das Modell EXP-005 erreichte eine scheinbar exzellente mAP50 von 82.3 %. Eine kritische Analyse zeigte jedoch, dass dies das Resultat von *Data Leakage* und *Background Overfitting* war. Da die Bounding Boxes mittels Canny-Edge-Erkennung vollautomatisch erzeugt wurden, fokussierte sich das Labeling auf die harte Tischkante anstatt auf den Müll. Das Modell lernte schlichtweg den Tisch als Objekt zu erkennen. Auf ungesesehenen, realistischen Szenen brach die Erkennung völlig zusammen.

Erst die konsequente Datenbereinigung und der Wechsel zu hochwertigen, menschlich oder durch Foundation Models (SAM, (Kirillov u. a., 2023)) annotierten Datensätzen lösten dieses Problem. Der 4-Wege-Vergleich (EXP-013 bis EXP-016) unterstreicht dies: Die Hinzunahme des Roboflow-Datensatzes (Roboflow, 2023) in EXP-014 steigerte die Erkennungsrate von Trash um +11,3 pp im Vergleich zur Baseline.

6.3 Klassenungleichgewicht und visuelle Diversität von Restmüll

Eine inhärente Schwäche vieler öffentlicher Datensätze, die auch in MIRAs aggregiertem Datensatz *mira_tnr* (EXP-014) sichtbar wird, ist das starke Klassenungleichgewicht (Class Imbalance). Während beispielsweise Plastik (1316 Instanzen im Validierungsset) überrepräsentiert ist, bleibt *Trash* notorisch unterrepräsentiert.

Zudem weist *Trash* eine extrem hohe intra-klassenspezifische visuelle Varianz auf – ein zerknülltes Taschentuch sieht völlig anders aus als eine gebrauchte Maske oder ein Apfelgriebsch. Dies erklärt, warum *Trash* in allen Modellen durchweg die schwächste Klasse (nur 26.9 % mAP50 im besten Modell) darstellt. Auch im Field-Benchmark bleibt Trash mit 54.0 % F1 klar hinter den Wertstoffklassen (84.9 %–90.7 %), was die nur teilweise Validierung von H3

unterstreicht. Eine zukünftige industrielle Umsetzung muss hier mit gezieltem Oversampling oder synthetischer Datengenerierung gegensteuern.

6.4 Plattformkompatibilität und statische Dimensionierung

Während der Modellkompression und des Deployments traten zwei signifikante Hürden auf:

1. **LiteRT-Windows-Compilerkonflikt:** Der Versuch, das trainierte PyTorch-Modell lokal unter Windows auf INT8 zu quantisieren, scheiterte an Inkompatibilitäten der Compiler-Schnittstellen von Ultralytics. Dieses Problem wurde durch die Auslagerung des Quantisierungsschritts in eine cloudbasierte Linux-Umgebung (Google Colab) gelöst.
2. **Statische TFLite-Tensorabmessungen:** Im Gegensatz zu dynamischen PyTorch-Modellen, die Eingabebilder flexibel skalieren, werden voll quantisierte TFLite-Modelle für eine feste Eingabematrix (z. B. 320×320 Pixel) kompiliert. Ein dimensionaler Konflikt zur Laufzeit wurde durch einen explizit parametrisierten Re-Export behoben, was zudem die Rechenlast auf der Edge-CPU deutlich senkte.

6.5 Einschränkung: Vierklassiges Klassifikationssystem (Stage A)

Eine wesentliche Limitation der Klassifikationsphase (Stage A) besteht darin, dass der initiale Machbarkeitsnachweis auf nur vier Wertstoffklassen (Glas, Metall, Papier, Plastik) beschränkt war. Die Sammelklasse *Trash* (Restmüll) wurde bewusst erst in der Detektionsphase (Stage B) eingeführt, um die Komplexität des Proof of Concept zu begrenzen. In der praktischen Anwendung bedeutet dies, dass das Klassifikationsmodell Restmüll zwingend einer der vier bekannten Klassen zuordnen muss, was zwangsläufig zu Fehlwürfen führt. Diese Einschränkung war als methodische Entscheidung jedoch vertretbar, da Stage A primär der Validierung des Transfer-Learning-Ansatzes diene. Für ein produktives System müsste entweder die Klassifikation um eine fünfte Klasse erweitert oder vollständig durch die Detektionsstufe ersetzt werden.

6.6 Einschränkung: Fehlende Edge-Hardware-Benchmarks

Die gemessenen Inferenzlatenzen in dieser Arbeit wurden ausschließlich auf einer Intel-Core-i7-CPU und einer NVIDIA-T4-Cloud-GPU erfasst. Die geplante Zielhardware (Raspberry Pi Zero 2W bzw. Raspberry Pi 4 mit Cortex-A53- bzw. Cortex-A72-Kernen) steht noch nicht als Testplattform zur Verfügung. Die erwarteten Latenzen von 90 ms bis 120 ms (ca. 8–11 FPS) basieren auf Compiler-Simulationen des LiteRT-INT8-Exports und sind nicht durch physikalische Messungen verifiziert. Für eine abschließende Bewertung der Edge-Tauglichkeit sind daher reale Benchmarks auf dem Raspberry Pi Zero 2W und dem Raspberry Pi 4 erforderlich. Die vorliegenden INT8-Quantisierungsergebnisse (2,90 MB Modellgröße) deuten jedoch stark auf eine ausreichende Performance hin.

6.7 Abwägung: INT8-Quantisierung versus Genauigkeit

Die Feldbenchmark-Ergebnisse zeigen einen F1-Verlust von 13 Prozentpunkten für EXP-014 durch die INT8-Quantisierung ($85,8\% \rightarrow 72,8\%$). Dieser Rückgang ist auf die diskrete ganzzahlige Rundung der Aktivierungsgewichte zurückzuführen, die feinkörnige Entscheidungsgrenzen vergrößert. Die Quantisierung ist dennoch für das Einsatzszenario sinnvoll, da sie die Modellgröße um den Faktor 3,5 reduziert ($10,14 \text{ MB} \rightarrow 2,90 \text{ MB}$) und den Einsatz auf CPUs ohne FPU-Beschleunigung ermöglicht. Der Trade-off zwischen 13 pp F1-Verlust und einer um den Faktor 3,5 verkleinerten Binärdatei ist für den angestrebten Einsatzzweck akzeptabel, da selbst das quantisierte Modell mit 72,8 % F1 praxistaugliche Sortierergebnisse liefert.

6.8 Abschließende Einordnung

Die Arbeit repräsentiert einen voll funktionsfähigen Software-Prototyp. Aus wissenschaftlicher Sicht ist es ein Vorteil, die reine Computer-Vision-Pipeline isoliert zu verifizieren, bevor die Varianz eines physikalischen Roboterarms das Signal-Rausch-Verhältnis stört. Die Integration auf der Zielhardware steht als nächster logischer Entwicklungsschritt an.

7 Fazit und Ausblick

7.1 Fazit

Das Projekt MIRA hat erfolgreich demonstriert, dass die Kombination aus Transfer Learning, gezielter Datensatz-Fusion und Post-Training-Quantisierung zu einem potenziell praxistauglichen, ressourceneffizienten KI-System für die automatische Wertstoffsortierung führt. Die drei zentralen Hypothesen H1–H3 sowie das Entwurfsziel E1 wurden durch die Ergebnisse im Wesentlichen bestätigt:

- **H1:** Transfer Learning mit MobileNetV2 übertraf das Scratch-CNN um +26,42 pp (87,42 % vs. 61,00 %).
- **H2:** INT8-Quantisierung reduzierte die Modellgröße um den Faktor 9 auf 2,61 MB bei vernachlässigbarem Genauigkeitsverlust (−0,07 pp) und 74,2% Latenzsenkung.
- **H3:** Der Übergang zur lokalisierten Objekterkennung (YOLO11n) ermöglicht die gleichzeitige Erkennung mehrerer Objekte im Kamerabild und erreicht eine mAP50 von 60,7% auf dem vereinten Datensatz (TACO + TrashNet + Roboflow). Die Hypothese wird als teilweise validiert bewertet, da die Objekterkennung funktioniert, die Trash-Klasse jedoch mit 26,9% mAP50 noch erheblichen Verbesserungsbedarf aufweist.
- **E1 (Entwurfsziel):** Der EMA-Filter ($\alpha = 0,15$) eliminiert erfolgreich Signalflickern ohne kritischen mechanischen Verzögerung (ca. 250 ms).

Der entscheidende Durchbruch gelang durch die Erkenntnis, dass Datenqualität und -vielfalt die Modellleistung stärker beeinflussen als reine Architektur-Optimierungen (Zha u. a., 2023). Der GIGO-Effekt beim Auto-Labeling (EXP-005) zwang zur Adaption professioneller Datensätze, was die Generalisierungsfähigkeit dramatisch verbesserte. Der 4-Wege-Datensatzvergleich (EXP-013 bis EXP-016) belegt, dass die Kombination aus TACO, TrashNet und Roboflow das beste generalisierbare Ergebnis liefert (60,7 % mAP50, 85,8 % F1 im Praxistest).

7.2 Ausblick

Aus den Ergebnissen ergeben sich folgende nächste Schritte zur Fertigstellung des physischen Gesamtsystems:

1. **Mechatronischer Aufbau:** Das System wird als 3-DOF-Roboterarm realisiert. Ein Raspberry Pi Zero 2W oder Raspberry Pi 4 führt die Bildverarbeitung aus und sendet die Objektkoordinaten via Serial-Protokoll an einen ESP32-S3-Mikrocontroller, der die Servomotoren (MG996R) präzise ansteuert. Ein Time-of-Flight-Lasersensor (VL53L0X) am Greifer misst die Z-Achse für den Greifvorgang.
2. **Modellkompression:** Durch Knowledge Distillation (Hinton, Vinyals und Dean, 2015) könnte ein kleineres Studenten-Netzwerk das Verhalten des YOLO11n-Modells imitieren und die Inferenzlatenz weiter reduzieren. Ergänzend sollte Pruning die Modellstruktur verschlanken.
3. **Erweiterung der Klassifikation:** Das fünfklassige System sollte um Biomüll, Elektroschrott oder Textilien erweitert werden. Um das Klassenungleichgewicht bei *Trash* zu lösen, ist eine Pipeline zur synthetischen Datengenerierung mittels Diffusionsmodellen vorgesehen.

4. **Cloud-IoT-Erweiterung:** Das lokale Streamlit-Dashboard wird um eine MQTT-Schnittstelle zur Fernüberwachung mehrerer dezentraler Sortiereinheiten erweitert.
5. **Online-Lernen:** Ein Feedback-Loop, der falsch klassifizierte Objekte in den Trainingsdatensatz zurückführt, soll die langfristige Robustheit des Systems erhöhen.

Die Kosten für ein solches System könnten bei Serienproduktion auf unter 100€ gesenkt werden. Insgesamt zeigt diese Arbeit, dass Deep Learning nicht nur in Cloud-Umgebungen, sondern auch auf extrem ressourcenbeschränkter Hardware praktikable Lösungen für reale Probleme liefern kann.

Quellen- und Literaturverzeichnis

- Ben-David, Shai u. a. (2010). „A Theory of Learning from Different Domains“. In: *Machine Learning* 79.1-2, S. 151–175.
- Bengio, Yoshua, Aaron Courville und Pascal Vincent (2013). „Representation Learning: A Review and New Perspectives“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8, S. 1798–1828.
- Deng, Lei u. a. (2020). „Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey“. In: *Proceedings of the IEEE* 108.4, S. 485–532.
- Hinton, Geoffrey, Oriol Vinyals und Jeff Dean (2015). „Distilling the Knowledge in a Neural Network“. In: *NeurIPS Workshop on Deep Learning*.
- Jocher, Glenn, Ayush Chaurasia und Jing Qiu (2023). *Ultralytics YOLO*. Softwareversion 8.0 / 11.0, Zugriff am 07. Juli 2026. URL: <https://github.com/ultralytics/ultralytics>.
- Kirillov, Alexander u. a. (2023). „Segment anything“. In: *arXiv preprint arXiv:2304.02643*.
- Krishnamoorthi, Raghuraman (2018). „Quantizing deep convolutional networks for efficient inference: A whitepaper“. In: *arXiv preprint arXiv:1806.08342*.
- LeCun, Yann, Yoshua Bengio und Geoffrey Hinton (2015). „Deep learning“. In: *Nature* 521.7553, S. 436–444.
- Neubeck, Alexander und Luc Van Gool (2006). „Efficient Non-Maximum Suppression“. In: *International Conference on Pattern Recognition*. IEEE, S. 850–855.
- Proenca, Pedro F. und Pedro Simoes (2020). „TACO: Trash Annotations in Context for Litter Detection“. In: *arXiv preprint arXiv:2003.06975*.
- Redmon, Joseph u. a. (2016). „You Only Look Once: Unified, Real-Time Object Detection“. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, S. 779–788.
- Ren, Shaoqing u. a. (2015). „Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks“. In: *Advances in Neural Information Processing Systems*, S. 91–99.
- Roboflow (2023). *Trash Detection Dataset*. Roboflow Universe, Trash Detection Dataset; Zugriff am 07. Juli 2026. URL: <https://universe.roboflow.com/>.
- Sandler, Mark u. a. (2018). „MobileNetV2: Inverted Residuals and Linear Bottlenecks“. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, S. 4510–4520.
- TensorFlow Authors (2024). *Transfer Learning and Fine-Tuning*. Zugriff am 07. Juli 2026. URL: https://www.tensorflow.org/tutorials/images/transfer_learning.
- Umweltbundesamt (2023). *Aufkommen und Verwertung von Verpackungsabfällen in Deutschland*. Zugriff am 07. Juli 2026. URL: <https://www.umweltbundesamt.de/daten/ressourcen-abfall/verwertung-entsorgung-ausgewaehlter-abfallarten/verpackungsabfaelle>.
- Wang, Xiang, Yujie Zhang, Ran Wang u. a. (2020). „A Survey on TinyML: Hardware, Software, and Applications“. In: *IEEE Communications Surveys & Tutorials* 24.4, S. 2127–2160.
- WaRP Project (2024). *Waste Recycling Plant Dataset (WaRP)*. Zugriff am 07. Juli 2026. URL: <https://github.com/AIRI-Institute/WaRP>.
- Yang, Mindy und Gary Thung (2016). *Classification of trash for recyclability status*. Techn. Ber. CS229 Project Report, Stanford University.
- Yosinski, Jason u. a. (2014). „How transferable are features in deep neural networks?“ In: *Advances in Neural Information Processing Systems*, S. 3320–3328.
- Zha, Daochen u. a. (2023). „Data-centric Artificial Intelligence: A Survey“. In: *arXiv preprint arXiv:2303.10158*.
- Zhang, Yifu u. a. (2022). „ByteTrack: Multi-Object Tracking by Associating Every Detection Box“. In: *arXiv preprint arXiv:2110.06864*.

Unterstützungsleistungen

- **Claude (Anthropic PBC, San Francisco, USA):** KI-Sprachmodell; Art der Unterstützung: Theoretische Strukturierung der physikalischen Hypothesen, Anpassung der Keras- und TFLite-Exportparameter, Code-Review und Optimierung der asynchronen OpenCV-Inferenzschleifen.
- **[Name des Betreuungslehrers], Betreuungslehrer, Gymnasium Mülheim an der Ruhr, Mülheim an der Ruhr:** Art der Unterstützung: Fachliche Begleitung und Beratung bei der Themenfindung sowie der Strukturierung der schriftlichen Arbeit.
- **Sparkassenstiftung Mülheim an der Ruhr, Mülheim an der Ruhr:** Stiftung; Art der Unterstützung: Sponsoring-Anträge für mechatronische Hardwarekomponenten eingereicht.

Die physische Datenerfassung der Recyclingobjekte, das Einrichten des lokalen PyCharm-Arbeitsbereichs, das eigenständige Ausführen des Modelltrainings in der Google Colab- und Kaggle-Cloud sowie die systematische Fehleranalyse (wie die eigenständige Entdeckung des RGB/BGR-Kanalfehlers und des GIGO-Effekts beim Auto-Labeling) erfolgten vollständig eigenständig durch den Jungforscher. Die genannten KI-gestützten Unterstützungsleistungen dienten ausschließlich als methodische Hilfsmittel; alle wissenschaftlichen Schlussfolgerungen, Hypothesenformulierungen und experimentellen Designentscheidungen wurden eigenständig getroffen.

Anhang

Klassenschema-Mapping

Tabelle 6: Klassenschema-Mapping der Rohdatensätze auf die 5 MIRA-Klassen

MIRA-Zielklasse	Auswahl remapperter Quellklassen (Roboflow / TACO)
Glass (ID 0)	Glass-bottles, broken-glass, jars, glass-cups
Metal (ID 1)	Drink-cans, food-cans, aerosols, aluminium-foil, metal-lids, pop-tabs
Paper (ID 2)	Cardboard-boxes, drink-cartons, egg-cartons, paper-bags, tissues, newspaper
Plastic (ID 3)	PET-bottles, plastic-bags, bubble-wrap, crisp-packets, plastic-cups, food-wrap
Trash (ID 4)	Batteries, cigarettes, food-waste, face-masks, blister-packs, unlabeled-litter